

TortoiseGit Installation & Basic Use

What is TortoiseGit? A Windows interface for Git that adds common Git commands directly to File Explorer's right-click menu.

1. Install TortoiseGit

Download TortoiseGit from the official website:

[Download TortoiseGit](#)

Install the correct version for your Windows computer. On most modern Windows 11 computers, this will be the 64-bit version.

Requirement: Git for Windows must already be installed before running the custom setup.

2. Before Running the Setup

The setup asks for your Git author identity: Name and Email. These values are stored with your commits. The email does not need to be a real inbox, but use the identity required by your company or Git server when applicable.

Important Do not enter a password in either field.

3. Run the Setup

Run the batch file located in the same folder as this guide:

```
Setup-TortoiseGit.bat
```

The setup checks Git, imports the supplied TortoiseGit settings, saves your global Name and Email, and can enable the Windows Classic right-click menu.

To restore the normal Windows 11 right-click menu later, run:

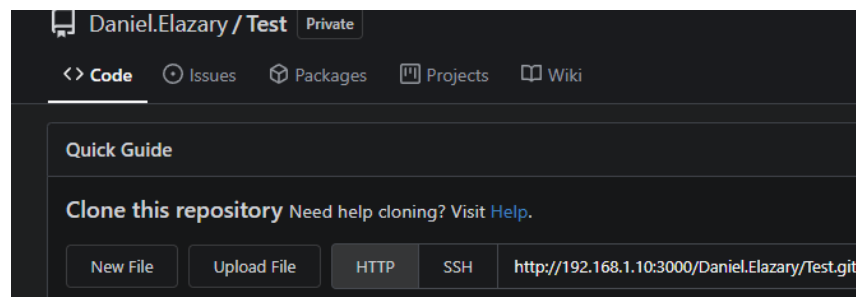
```
Batch-Dependencies\un-Classic-Windows-Menu.bat
```

The menu change may require restarting Windows Explorer or signing out and back in before it appears.

4. Basic TortoiseGit Workflow

Create or Copy the Online Repository URL

If the project uses a remote server such as Gitea, create or open the online repository and copy its HTTP repository URL.



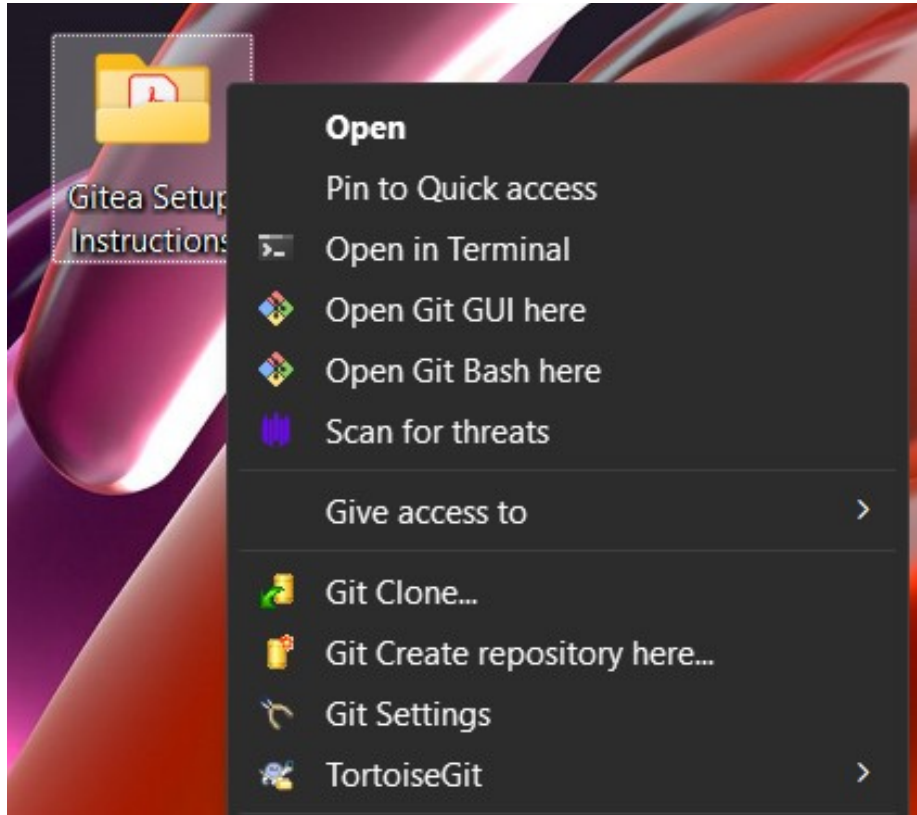
Copy the remote HTTP repository URL.

Create a Local Repository

Right-click the main project folder and select:

Git Create repository here...

Make sure **Make it Bare** is NOT selected, then click OK.



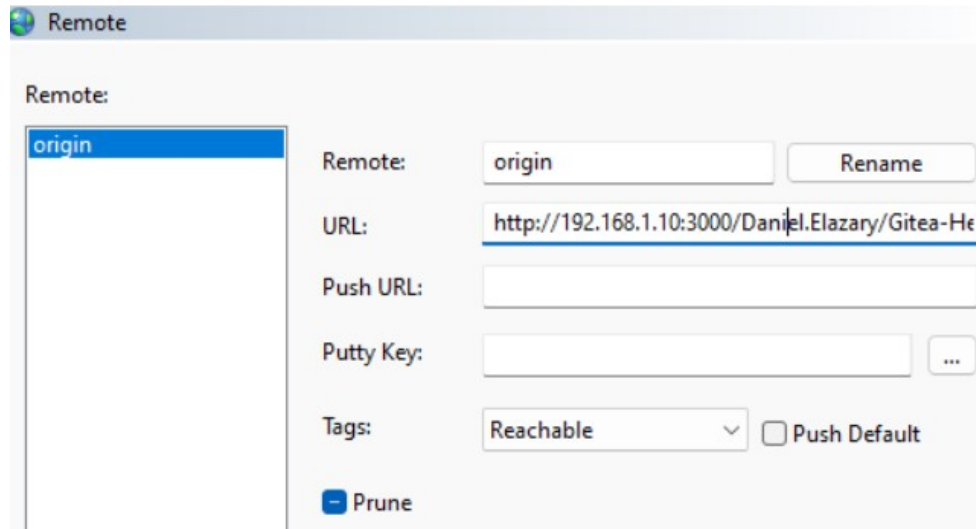
Windows right-click menu showing Git Create repository here.

Result The selected project folder is now a local Git repository. You can begin adding files, committing changes, and connecting the repository to a remote server.

Connect the Local Repository to the Server

Right-click the repository folder and select Git Fetch, then open Manage Remotes. Paste the repository URL into the URL field and use:

Remote name: origin



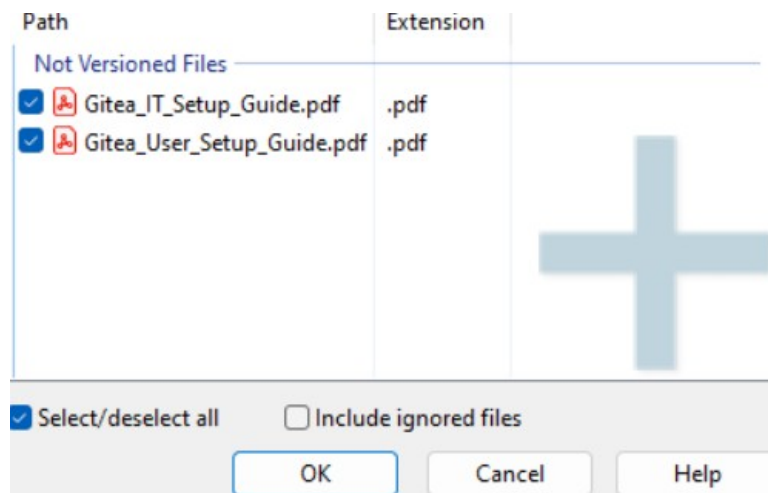
Manage Remotes window with the origin URL configured.

Add New Files

New files must be added before they can be committed. Right-click the repository folder and select:

Git Add...

Select the files you want Git to track and click OK.



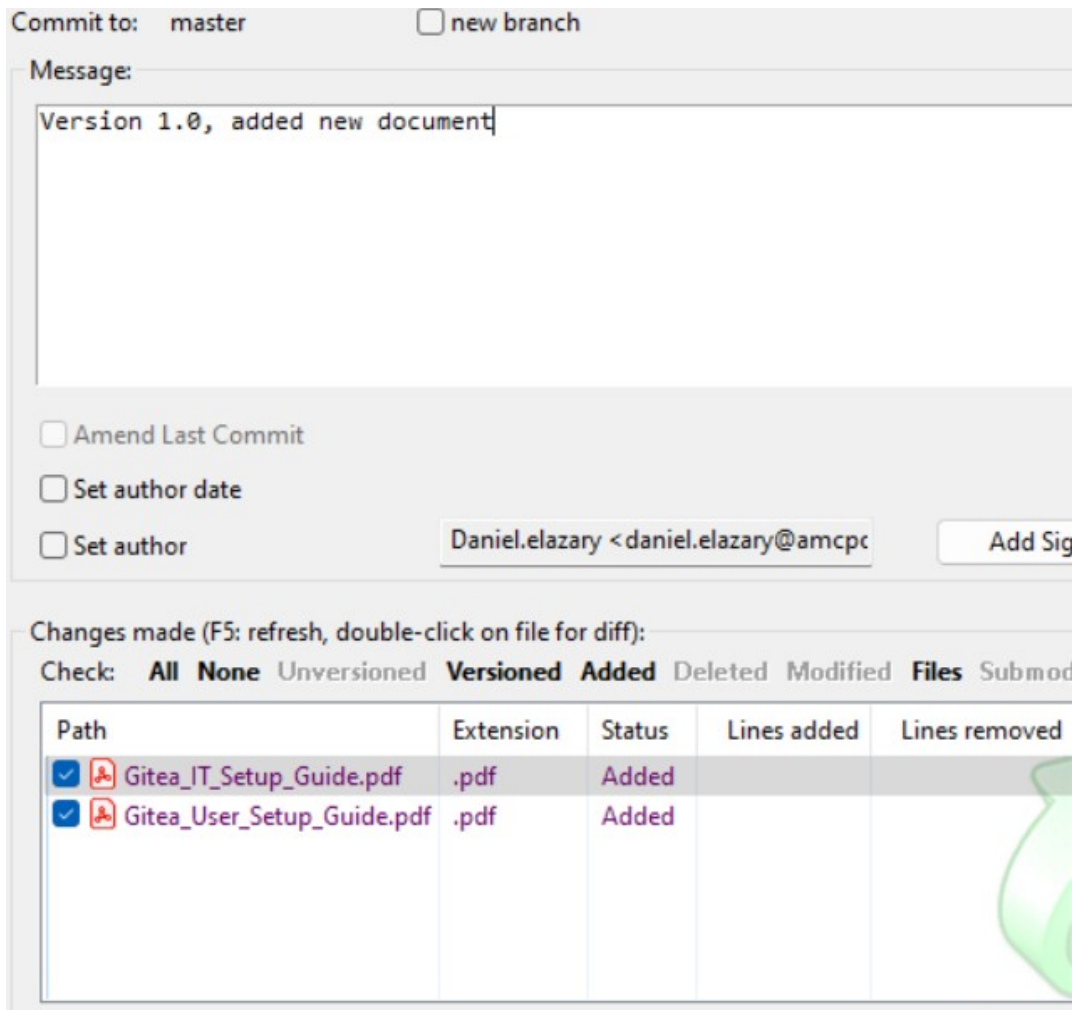
Git Add window used to select new files for tracking.

Commit Changes

A commit saves a version of your changes locally. Right-click the repository folder and select:

Git Commit -> "branch name"...

Enter a short message describing the change, select the intended files, and click OK.



Commit window showing the message field and changed files.

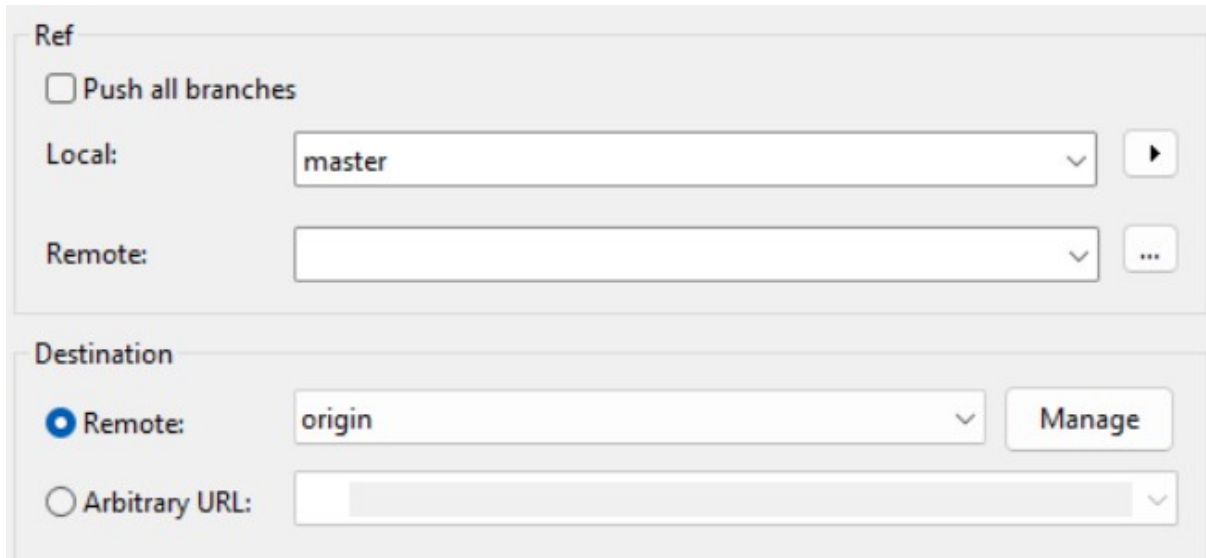
Remember A commit is stored locally on your computer. It is not sent to the remote server until you push it.

Push Changes

A commit is only stored on your computer until it is pushed. Right-click the repository and select:

Git Push...

The remote will normally be "origin". Click OK to send the committed changes to the server.



The screenshot shows the 'Git Push' dialog box. Under the 'Ref' section, the 'Push all branches' checkbox is unchecked. The 'Local:' dropdown is set to 'master'. The 'Remote:' dropdown is empty. Under the 'Destination' section, the 'Remote:' radio button is selected, and the dropdown is set to 'origin'. There is a 'Manage' button next to it. The 'Arbitrary URL:' radio button is unselected, and its dropdown is empty.

Git Push window with origin selected as the remote destination.

Typical sequence Edit files -> Add new files -> Commit -> Push

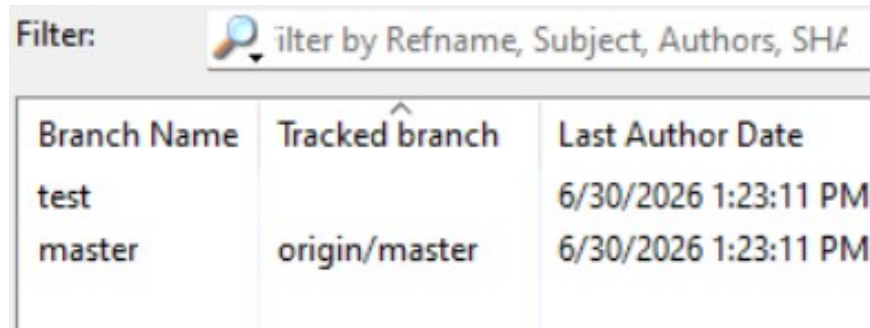
If a file was already tracked by Git, you normally do not need to use Git Add again just because the file was edited. The Add step is especially important for new, previously untracked files.

Branches

Branches let you develop or test changes separately without immediately changing the main branch. Use:

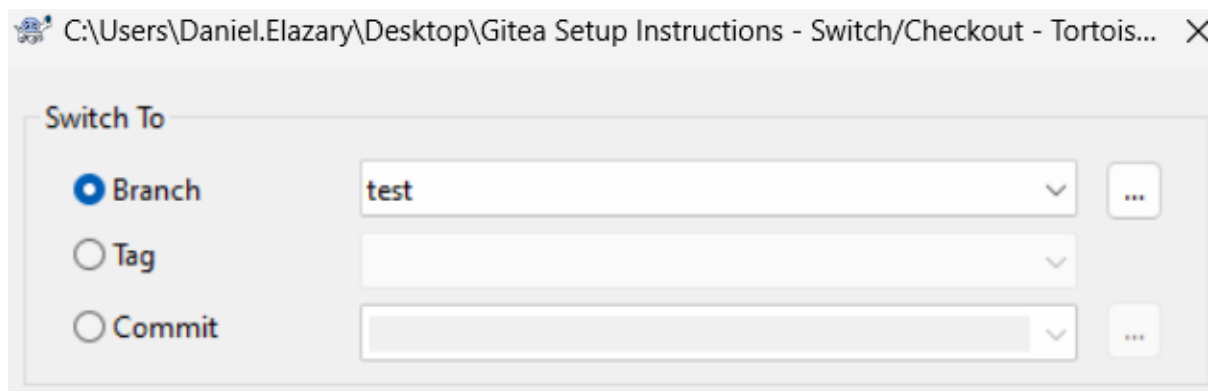
Git Switch/Checkout...

Create a new branch or choose an existing branch to work on.



Branch Name	Tracked branch	Last Author Date
test	origin/master	6/30/2026 1:23:11 PM
master	origin/master	6/30/2026 1:23:11 PM

Branch list showing available local and tracked branches.



Switch/Checkout window used to move to another branch.

Good practice Before changing branches, make sure important work has been committed or otherwise safely saved.

Merge a Branch

First switch to the branch that should RECEIVE the finished changes. Then right-click the repository and select:

Git Merge...

Choose the branch containing the changes and complete the merge.

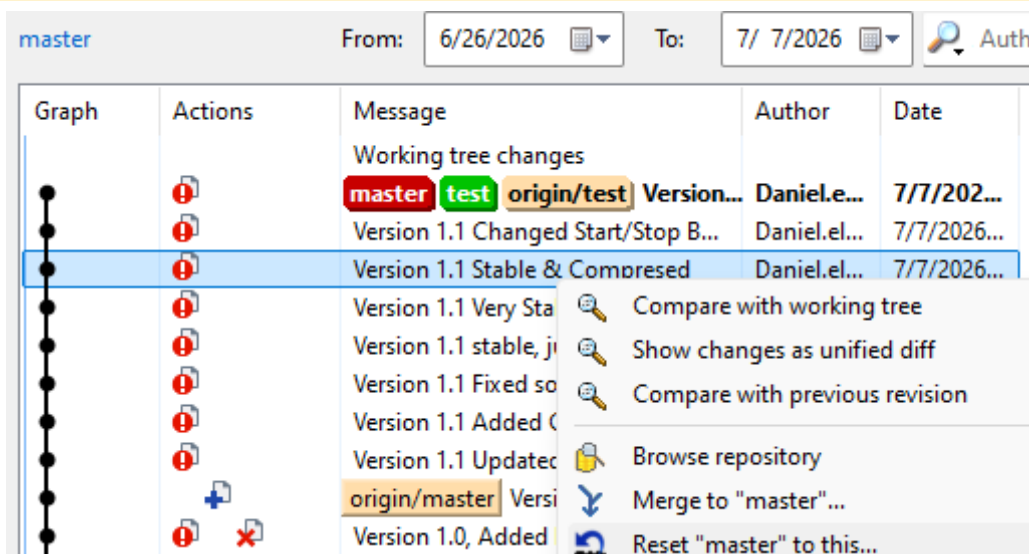


Merge window showing the branch selected as the source of changes.

Restore an Older Version

Right-click the repository and select Git Show Log. Find the older commit, right-click it, and choose the appropriate reset option.

Warning Resetting can discard newer work. Back up anything important first.



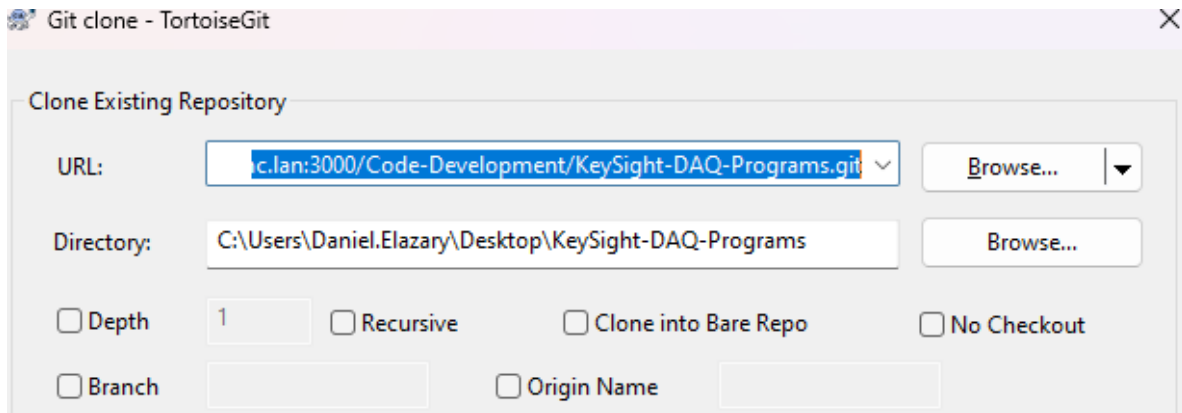
Show Log view with the Reset to this commit option.

Clone an Existing Repository

To download an existing repository to a new computer, copy its HTTP URL, right-click where you want the folder, and select:

Git Clone...

Paste the URL, choose the destination folder, and click OK.



Git Clone window showing the repository URL and local destination.

5. Normal Daily Use

For an existing project, the most common workflow is:

Edit Files -> Git Add new files -> Commit -> Push

Use branches when you want to develop or test changes separately from the main working branch.

Quick reference Create local repository -> Connect origin -> Add new files -> Commit -> Push. For an existing repository on a new computer, use Git Clone instead of creating a new local repository.